Task: Build a Predictive Model and Show it as a Web Application Using Streamlit on localhost

Objective

The goal of this task is to create a complete data science project by building a predictive model, performing exploratory data analysis (EDA), training and evaluating the model, and showing it as a web application using Streamlit on localhost. The goal is to apply the concepts you learned throughout the course and demonstrate your understanding by building a complete data science project from scratch.

Steps

1. Data Collection and Exploration

- **Load the Dataset:**
  - Use Pandas to load the dataset into a DataFrame.
- **Initial Exploration:**
  - Examine the dataset structure, data types, and summary statistics.
  - Identify potential issues such as missing values, inconsistent data, or outliers.

2. Data Cleaning and Transformation

- **Handle Missing Values:**
  - Apply appropriate techniques such as imputation or removal.
- **Outlier Detection and Treatment:**
  - Use statistical methods or visualization techniques to identify and handle outliers.
- **Data Transformation:**
  - Encode categorical variables using methods like one-hot encoding or label encoding.
  - Normalize or scale numerical features as needed.
  - Create new features if necessary to improve model performance.

3. Exploratory Data Analysis (EDA)

- **Visualization:**
  - Use tools such as Matplotlib and Seaborn to analyze data distributions, correlations, and trends.
  - Explore relationships between variables and identify patterns.
- **Key Findings:**
  - Summarize insights obtained from the visualizations and statistical analysis.

4. Feature Selection

- **Relevance Analysis:**
  - Use correlation analysis, feature importance metrics, or domain knowledge to identify the most relevant features.
- **Dimensionality Reduction:**
  - Remove irrelevant or redundant features to simplify the model.

5. Model Development

- **Data Splitting:**
  - Divide the dataset into training and testing sets.
- **Algorithm Selection:**
  - Choose a machine learning algorithm suitable for the problem (e.g., Linear Regression, Logistic Regression, KNN, Naïve Bayes,etc).
- **Model Training:**
  - Train the model using the training data.
- **Model Evaluation:**
  - Validate the model on the testing data using appropriate performance metrics depending on the problem type. (e.g., accuracy, precision, recall, F1-score,MSE, MAE, RMSE).

6. Model Evaluation and Hyperparameter Tuning

- **Cross-Validation:**
  - Perform cross-validation to assess the model's performance.
- **Hyperparameter Tuning:**
  - Optimize model parameters using techniques like Grid Search or Random Search.
- **Final Evaluation:**

    - Compare models and select the best-performing one based on evaluation metrics.

7. Model Deployment

- **Streamlit Deployment on localhost:**
    - Build an interactive web application using Streamlit on localhost.
    - Create a user-friendly interface to accept user inputs and display model predictions.

8. Documentation

- **Process Documentation:**
    - Document the entire project, including:
        - Dataset description
        - Data cleaning and transformation steps
        - EDA insights
        - Model development, evaluation, and deployment process
- **Code Documentation:**
    - Include comments in your code to explain each step clearly.

9. Version Control

- **Git Integration:**
    - Track project progress using Git.
    - Create a GitHub repository to store your project code.
- **Repository Organization:**
    - Ensure the repository contains:
        - Project code
        - A README file explaining the project and instructions to run the application

---

Deliverables

1. **Jupyter Notebook or Python Script:**
    - Contains:
        - Data loading and cleaning steps
        - EDA visualizations and summaries
        - Feature selection process

        - Model training, evaluation, and tuning
2. **Streamlit Web Application on localhost:**
    - Interactive application for user inputs and predictions.
3. **GitHub Repository:**
    - Includes:
        - Project code
        - Documentation
        - README file

Bonus (Optional)

1. **Additional Models:**
    - Implement multiple machine learning models and compare their performance.
2. **REST API Deployment:**
    - Deploy the model using Flask to create a REST API for predictions.
3. **Containerization:**
    - Use Docker to containerize the application for easy deployment on any server.

Good luck, and enjoy applying your data science skills to solve a real-world problem!